SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: CRYPTOGRAPHY AND NETWORK SECURITY
COURSE CODE: CSA5117

COURSE OUTCOME COVERED

CO2: Analyze Public Key crypto system strategies using RSA and key Exchange for Encryption algorithm to strengthen information security. (BL4,BL5)

Assessment Tool Weightage: 10%

QUESTIONS: **Total Marks: 50**

Annexure A

Debugging & Optimization Task

Code of Conduct:

I L. Infant Ency (196526003) *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A
Debugging & Optimisation Task

1. Debugging & Optimization of Symmetric Encryption (DES / 3DES Implementation)
A Python program implementing **DES and 3DES encryption** produces inconsistent ciphertext for identical inputs across multiple runs.

A. Identify and fix logical and implementation errors in key scheduling and permutation steps.
B. Optimize the code to reduce execution time for large input datasets (≥10 MB).
C. Justify whether replacing DES/3DES with a modern cipher improves both security and performance.

2. Performance Analysis of Block Cipher Modes of Operation A Python script implements **AES-like block cipher modes (ECB, CBC, CFB, OFB)** but suffers from high latency and memory inefficiency.

A. Debug incorrect padding and IV handling causing decryption failures.
B. Refactor the code to minimize redundant computations and memory usage.
C. Compare and optimize the implementation to determine the most efficient mode for real-time secure communication.

3. Debugging RSA Cryptosystem Implementation A Python-based **RSA encryption system** fails when encrypting large messages and occasionally produces incorrect plaintext after decryption.

A. Identify issues in prime generation, key generation, and modular exponentiation.
B. Optimize the algorithm using efficient techniques (e.g., fast exponentiation, Chinese Remainder Theorem).
C. Evaluate the trade-offs between key size, security, and computational performance.

4. Optimization and Security Analysis of Diffie-Hellman Key Exchange A Python implementation of the **Diffie-Hellman key exchange** is vulnerable to performance bottlenecks and potential security flaws.

A. Debug incorrect shared key generation due to improper modular arithmetic.
B. Optimize exponentiation operations for large prime values.
C. Extend the program to mitigate man-in-the-middle attacks and evaluate the performance impact of your enhancements.

5. Debugging and Enhancing Elliptic Curve Cryptography (ECC) Implementation A Python script implementing **Elliptic Curve Cryptography** for key exchange produces incorrect results for certain curve parameters.

A. Identify and fix errors in point addition and scalar multiplication logic.
B. Optimize the implementation using efficient algorithms (e.g., double-and-add).

C. Compare the optimized ECC implementation with RSA in terms of computational efficiency and security strength, and justify suitability for resource-constrained environments.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Identification	20%	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	25%	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues
Optimization	20%	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained
Analysis	20%	Demonstrates strong logical reasoning and clear justification of solutions	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrated
Code Quality	15%	Produces clean, well-structured code with proper comments and documentation	Code is mostly clear with minor documentation gaps	Code lacks structure and proper documentation	Poorly written code with no documentation

CO2 AT3

Assessment Tool-3

1. Infant Ency
1961526 00003

Course Name: Cryptography
Course Code: CSA5117

2) Performance Analysis of Block cipher Modes of operation:

A) Debug incorrect Padding and IV handling causing decryption failures.

Block Cipher modes such as ECB, CBC, CFB and OFB are used for encryption.

=> Padding: Extra bits are added when Plaintext does not match the required block size

=> IV (Initialization Vector) used in CBC, CFB, and OFB modes for proper encryption and decryption

=> ECB. Does not require an IV but reveals data Patterns

=> Solution: use correct Padding and proper IV handling to avoid decryption errors

B) Refactor the code to minimize redundant computations and memory usage

*Code optimization improves execution speed and reduces memory usage

=> Process large files in smaller chunks

=> Avoid unnecessary data copying and repeated calculations

=> use optimized libraries like Python's cryptography

=> Release unused memory

Result:
These methods improve performance and reduce memory consumption

c) compare and optimize the implementation for real time secure communication

Mode	Advantages	Limitations
=> ECB	=> Fast and Parallel	=> Reveals data Patterns
=> CBC	=> Provide confidentiality	=> sequential encryption
=> CFB	Processes smaller data units	Sequential encryption
=> OFB	Generates a continuous keystream	IV reuse is dangerous.

conclusion:

⇒ ECB is not Recommended for Secure communication

⇒ CBC, CFB and OFB Require correct IV handling

⇒ AES - GCM is Recommended for modern real-time communication because it Provides both encryption and data authentication.